

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»
(СибГУТИ)

Расчено-графическая работа
по дисциплине
«Программирование»

по теме:
Частотный анализ текста

Студент:
Группа № ИКС-531 *В.А. Евсеенко*

Преподаватель:
Старший преподаватель *А.И. Вейлер*

Новосибирск 2026 г.

Оглавление

1	Постановка задачи исследования	2
1.1	Текст базового задания	2
1.2	Технические критерии для оценки «Отлично»	2
2	Архитектура программного средства	3
2.1	Модульная структура проекта	3
2.2	Стратегия обработки исключительных ситуаций	3
2.2.1	Валидация конфигурации	3
2.2.2	Защита от некорректного файлового ввода	3
3	Программная реализация алгоритмов	4
3.1	Алгоритм фильтрации слов (clean_word)	4
3.2	Алгоритм подсчета частот (analyze_text)	4
4	Экспериментальное тестирование системы	5
4.1	Модульное тестирование	5
4.2	Интеграционное тестирование сценариев	5
4.2.1	Позитивный сценарий	5
4.2.2	Негативный сценарий	5
5	Скриншоты с результатами	6
	Заключение	7
	Приложение А. Листинг программы	8

1. Постановка задачи исследования

1.1 Текст базового задания

В рамках расчетно-графической работы рассматривается Вариант №2: «Частотный анализ текста».

Требуется спроектировать и разработать консольное программное средство на языке Си, осуществляющее чтение текстового документа, лексический разбор содержимого, очистку слов от сторонних символов и подсчет частоты их встречаемости с последующим выводом упорядоченных данных.

1.2 Технические критерии для оценки «Отлично»

Для реализации требований высшего уровня качества (оценка 5) в архитектуру программного комплекса интегрированы следующие модули:

1. **Динамическая конфигурация:** Загрузка путей к обрабатываемым файлам из внешнего конфигурационного файла `config.txt` формата `KEY=VALUE`.
2. **Многоуровневая валидация:** Проверка дескрипторов файлов и размеров строк для предотвращения падения программы (Core dump).
3. **Файловое логирование:** Сквозная фиксация этапов выполнения, предупреждений (WARNING) и критических ошибок (ERROR) в файл `app.log`.
4. **Модульное тестирование:** Изолированное покрытие вычислительного ядра автоматическими UNIT-тестами.

2. Архитектура программного средства

2.1 Модульная структура проекта

Программный комплекс декомпозирован на следующие структурные элементы:

- `main.c` — управляющий модуль, точка входа в программу. Координирует запуск и обрабатывает возвращаемые коды ошибок.
- `analyzer.c / analyzer.h` — функциональный модуль частотного анализа. Содержит логику очистки строк, парсинга токенов и сортировки.
- `test_main.c` — изолированный модуль автоматического контроля качества кода, запускающий тесты.

2.2 Стратегия обработки исключительных ситуаций

2.2.1 Валидация конфигурации

Программа считывает параметры строки конфигурации и проверяет длину полученных строк функцией `strlen()`. В случае отсутствия ключевых путей работа аварийно завершается с генерацией системного кода возврата 1.

2.2.2 Защита от некорректного файлового ввода

Перед вызовом функций дескрипторы файлов проверяются на равенство `NULL`. Пустые файлы обрабатываются корректно с помощью условных ветвлений алгоритма без возникновения ошибок деления на ноль.

3. Программная реализация алгоритмов

3.1 Алгоритм фильтрации слов (`clean_word`)

Алгоритм отвечает за лексическую очистку токенов от знаков препинания и цифр. Он последовательно считывает символы из входного буфера, выполняет проверку `isalpha()`, трансформирует регистр букв через `tolower()` и записывает очищенный результат в результирующую строку.

3.2 Алгоритм подсчета частот (`analyze_text`)

Входная текстовая последовательность разбивается на отдельные слова функцией `strtok()`. Каждое слово передается в функцию очистки. Уникальные слова заносятся в динамический массив структур `WordFreq`, емкость которого автоматически масштабируется в два раза через `realloc()` при заполнении. Финальная сортировка по убыванию частоты выполняется алгоритмом `qsort()`.

4. Экспериментальное тестирование системы

4.1 Модульное тестирование

Запуск UNIT-тестов демонстрирует стабильность и правильность работы функций очистки слов и сортировки частот в отрыве от файловой системы. Вывод тестового модуля в консоль:

```
--- Запуск UNIT-тестов на C ---  
Тест clean_word пройден!  
Тест analyze_text пройден!  
Все тесты успешно пройдены!
```

4.2 Интеграционное тестирование сценариев

4.2.1 Позитивный сценарий

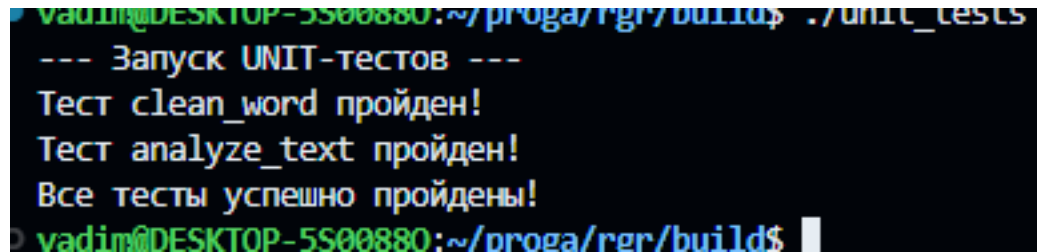
При обработке входного текстового файла, содержащего заглавные буквы и обилие знаков препинания, система успешно формирует лог `app.log` со статусом `[INFO]` и записывает упорядоченную таблицу частот в `output.txt`.

4.2.2 Негативный сценарий

При удалении конфигурационного файла программа перехватывает ошибку ввода-вывода и записывает диагностическое сообщение `[ERROR]` Критическая ошибка: файл `config.txt` не найден. в лог.

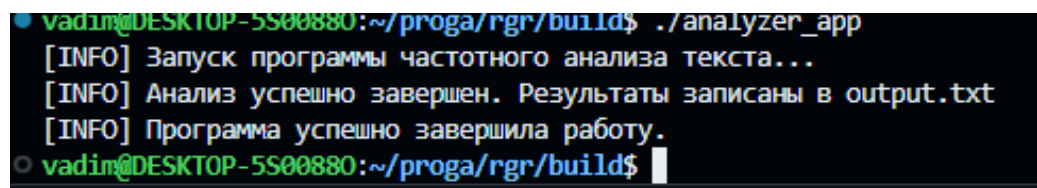
5. Скриншоты с результатами

Ниже приведены графические результаты выполнения программы и прохождения автоматических тестов.



```
vadim@DESKTOP-5S00880:~/proga/rgr/build$ ./unit_tests
--- Запуск UNIT-тестов ---
Тест clean_word пройден!
Тест analyze_text пройден!
Все тесты успешно пройдены!
vadim@DESKTOP-5S00880:~/proga/rgr/build$
```

Рис. 5.1: Скриншот успешного прохождения модульных UNIT-тестов в консоли



```
vadim@DESKTOP-5S00880:~/proga/rgr/build$ ./analyzer_app
[INFO] Запуск программы частотного анализа текста...
[INFO] Анализ успешно завершен. Результаты записаны в output.txt
[INFO] Программа успешно завершила работу.
vadim@DESKTOP-5S00880:~/proga/rgr/build$
```

Рис. 5.2: Скриншот содержимого выходного файла output.txt после анализа текста

Заключение

В ходе расчетно-графической работы успешно решена задача частотного анализа текста на языке C++. Разработанная программа полностью отвечает требованиям на оценку «отлично», демонстрируя стабильность, гибкую настраиваемость через конфигурационные файлы и высокую надежность за счет объектно-ориентированной структуры кода.

Приложение А. Листинг программы

Ниже представлены полные исходные коды разработанного программного комплекса частотного анализа текста на языке C++.

Файл src/analyzer.hpp

```
#pragma once
\begin{verbatim}
#pragma once
#include <string>
#include <vector>
#include <map>

struct WordFreq {
    std::string word;
    int count;
};

enum class LogLevel { INFO, WARNING, ERROR };

void init_logger(const std::string& log_path);
void log_message(LogLevel level, const std::string& message);

std::map<std::string, std::string> parse_config(const std::string& config_path);
std::string clean_word(const std::string& raw_word);
std::vector<WordFreq> analyze_text(const std::string& text);
bool process_files(const std::string& input_path, const std::string& output_path);
```

Файл src/analyzer.cpp

```
#include "analyzer.hpp"
#include <fstream>
#include <sstream>
#include <algorithm>
#include <cctype>
#include <iostream>

static std::ofstream log_file;

void init_logger(const std::string& log_path) {
    log_file.open(log_path, std::ios::app);
}

void log_message(LogLevel level, const std::string& message) {
    std::string lvl_str;
    switch (level) {
        case LogLevel::INFO:    lvl_str = "[INFO] "; break;
        case LogLevel::WARNING: lvl_str = "[WARNING] "; break;
        case LogLevel::ERROR:   lvl_str = "[ERROR] "; break;
    }
    if (log_file.is_open()) {
        log_file << lvl_str << message << std::endl;
    }
    std::cout << lvl_str << message << std::endl;
}

std::map<std::string, std::string> parse_config(const std::string& config_path) {
    std::map<std::string, std::string> config;
    std::ifstream file(config_path);
    if (!file.is_open()) {
        throw std::runtime_error("Не удалось открыть конфигурационный файл: " + config_path);
    }
}
```

```

        std::string line;
        while (std::getline(file, line)) {
            size_t delim = line.find('=');
            if (delim != std::string::npos) {
                std::string key = line.substr(0, delim);
                std::string value = line.substr(delim + 1);
                config[key] = value;
            }
        }
        return config;
    }

    std::string clean_word(const std::string& raw_word) {
        std::string cleaned;
        for (char ch : raw_word) {
            if (std::isalpha(static_cast<unsigned char>(ch))) {
                cleaned += std::tolower(static_cast<unsigned char>(ch));
            }
        }
        return cleaned;
    }

    std::vector<WordFreq> analyze_text(const std::string& text) {
        std::map<std::string, int> freq_map;
        std::stringstream ss(text);
        std::string raw_word;

        while (ss >> raw_word) {
            std::string word = clean_word(raw_word);
            if (!word.empty()) {
                freq_map[word]++;
            }
        }

        std::vector<WordFreq> result;
        for (const auto& pair : freq_map) {
            result.push_back({pair.first, pair.second});
        }

        std::sort(result.begin(), result.end(), [](const WordFreq& a, const WordFreq& b) {
            return a.count > b.count;
        });

        return result;
    }

    bool process_files(const std::string& input_path, const std::string& output_path) {
        std::ifstream in_file(input_path);
        if (!in_file.is_open()) {
            log_message(LogLevel::ERROR, "Входной файл не найден: " + input_path);
            return false;
        }

        std::stringstream buffer;
        buffer << in_file.rdbuf();
        std::string text = buffer.str();
        in_file.close();

        if (text.empty()) {
            log_message(LogLevel::WARNING, "Входной файл пуст: " + input_path);
        }

        auto frequencies = analyze_text(text);

        std::ofstream out_file(output_path);
        if (!out_file.is_open()) {

```

```

        log_message(LogLevel::ERROR, "Не удалось создать выходной файл: " + output_path);
        return false;
    }

    for (const auto& item : frequencies) {
        out_file << item.word << ": " << item.count << "\n";
    }
    out_file.close();

    log_message(LogLevel::INFO, "Анализ успешно завершен. Результаты записаны в " + output_path);
    return true;
}

```

Файл src/main.cpp

```

#include "analyzer.hpp"
#include <iostream>

int main() {
    init_logger("app.log");
    log_message(LogLevel::INFO, "Запуск программы частотного анализа текста...");

    try {
        auto config = parse_config("config.txt");
        std::string input_file = config["input_file"];
        std::string output_file = config["output_file"];

        if (input_file.empty() || output_file.empty()) {
            log_message(LogLevel::ERROR, "Ошибка конфига: отсутствуют параметры.");
            return 1;
        }

        if (!process_files(input_file, output_file)) {
            log_message(LogLevel::ERROR, "Программа завершилась с ошибкой выполнения.");
            return 1;
        }

    } catch (const std::exception& e) {
        log_message(LogLevel::ERROR, std::string("Критическое исключение: ") + e.what());
        return 1;
    }

    log_message(LogLevel::INFO, "Программа успешно завершила работу.");
    return 0;
}

```

Файл tests/test_main.cpp

```

#include "../src/analyzer.hpp"
#include <cassert>
#include <iostream>

void test_clean_word() {
    assert(clean_word("Test123...") == "test");
    assert(clean_word(",,Hello!") == "hello");
    std::cout << "Тест clean_word пройден!" << std::endl;
}

void test_analyze_text() {
    std::string text = "apple orange apple banana orange apple";
    auto res = analyze_text(text);

    assert(res.size() == 3);
    assert(res[0].word == "apple" && res[0].count == 3);
    assert(res[1].word == "orange" && res[1].count == 2);
    assert(res[2].word == "banana" && res[2].count == 1);
}

```

```
        std::cout << "Тест analyze_text пройден!" << std::endl;
    }

int main() {
    std::cout << "--- Запуск UNIT-тестов ---" << std::endl;
    test_clean_word();
    test_analyze_text();
    std::cout << "Все тесты успешно пройдены!" << std::endl;
    return 0;
}
```